

[66] Why You Should Run TPC-DS: A Workload Analysis

요약

본 논문은 2007 년 VLDB 에서 발표된 Poess, Nambiar, Walrath 의 연구로, TPC-DS 벤치마크가 현대 데이터 웨어하우스 워크로드를 더 현실적으로 반영하는 이유를 설명한다. TPC-H 가 분석 쿼리의 기본이라면, TPC-DS 는 더 복잡하고 다양한 현실 세계의 데이터 분석을 모사한다.

TPC-DS vs TPC-H 비교:

TPC-H 의 한계:

- 단일 도메인(판매) 초점
- 비교적 단순한 스키마
- 정적인 데이터 세트

TPC-DS 의 특징:

- 다양한 판매 채널 (온라인, 오프라인, 카탈로그)
- 더 복잡한 비즈니스 로직
- 현실적인 데이터 분포 편향
- 다양한 쿼리 타입 (OLAP, 텍스트 검색 요소 등)

TPC-DS 의 99 개 쿼리:

- 다양한 분석 패턴
- 복합 필터링과 조인
- 윈도우 함수, 시계열 분석
- 자체 조인(self-join) 등 다양한 기법

현실성 증대:

- 실제 소매점 운영에서 필요한 분석 반영
- 데이터의 비균등 분포
- 계절성, 트렌드 분석의 필요성

Exqutor 에서 TPC-DS 를 고려할 때, 벡터 검색을 추가한 확장 벤치마크가 더욱 현실적이 될 수 있으며, 다양한 실제 워크로드를 반영하는 평가가 가능하다.

상세분석

66.1 TPC-H 의 한계와 TPC-DS 의 필요성

1.1 TPC-H 의 설계 목표와 한계

설계 목표:

- SQL 쿼리 성능 비교의 표준화
- 상대적으로 단순하고 이해하기 쉬운 구조
- 재현 가능한 실험 환경

현실과의 차이:

1. 단일 도메인: 항공 우주 부품 공급망
2. 균등 분포: 랜덤하게 생성된 데이터
3. 단순 스키마: 8 개 테이블만 사용
4. 정적 워크로드: 고정된 22 개 쿼리

1.2 실제 데이터 웨어하우스의 특징

다양한 데이터 출처:

- POS(판매점) 데이터
- 온라인 주문 데이터
- 고객 관계 정보
- 재고 정보
- 마케팅 캠페인 데이터

비균등 분포:

- 인기 상품은 자주, 비인기 상품은 드물게 팔림
- 특정 시간(휴가)에 판매 집중
- 지역별 선호도 편차

복잡한 비즈니스 로직:

- 다중 채널 데이터 통합
- 반품, 교환 처리
- 프로모션 효과 분석

66.2 TPC-DS 의 구조와 특징

2.1 스키마의 복잡성

TPC-DS 테이블 구성 (99 개 중 일부):

고객 관련:

- customer: 기본 고객 정보
- customer_address: 배송/청구 주소
- customer_demographics: 인구 통계

판매 채널:

- store: 오프라인 매장 정보
- web_site: 온라인 판매 채널
- catalog_page: 카탈로그 페이지

거래 데이터:

- store_sales: 매장 판매
- web_sales: 온라인 판매
- catalog_sales: 카탈로그 판매

상품 관련:

- item: 상품 기본 정보
- inventory: 재고
- promotion: 프로모션

총 99 개 테이블 vs TPC-H 의 8 개

2.2 데이터의 현실성

Zipf 분포 (현실의 히트상품 패턴):

- 소수의 상품이 대부분 판매
- 다수의 상품이 거의 팔리지 않음
- 키워드 검색처럼 장꼬리 분포

계절성 데이터:

- 특정 시즌(크리스마스, 여름) 판매 급증
- 요일별 패턴 (주말 증가)
- 휴일 효과

지역별 편차:

- 도시 vs 시골 지역의 선호도 차이
- 문화권별 상품 선호도
- 배송비 등의 지역별 차별화

66.3 TPC-DS 의 99 개 쿼리 특성

3.1 쿼리 복잡도 분류

Type A: 단순 쿼리 (약 30 개)

- 1-2 개 테이블, 간단한 WHERE 절
- 기본 집계 함수

Type B: 중간 복잡도 (약 50 개)

- 3-5 개 테이블 조인
- 복합 필터링
- 윈도우 함수

Type C: 고복잡도 (약 19 개)

- 6 개 이상 테이블 조인
- 자체 조인(self-join) 포함
- 상관 서브쿼리
- CTE(Common Table Expression) 사용

3.2 분석 패턴

시계열 분석:

Q11: 작년 대비 올해 고객 성장률

- 연도별 비교
- CASE 문과 윈도우 함수

마켓 바스켓 분석:

Q33: 함께 구매되는 상품 조합

- 자체 조인으로 같은 고객의 구매 패턴
- 상관관계 분석

고객 분할:

Q48: 구매 패턴별 고객 세분화

- 다중 조인
- 복합 그룹화

상품 추천:

Q20: 자주 함께 팔리는 상품

- 판매 데이터의 연관성 분석

3.3 현실성이 높은 이유

비즈니스 질문과의 직결:

- "이번 분기 판매 추이는?"
- "어떤 제품이 인기인가?"
- "고객 만족도는?"
- "마케팅 효과는?"

이 질문들이 TPC-DS 쿼리에 반영됨

66.4 TPC-H 와 TPC-DS 의 성능 차이

4.1 쿼리 복잡도의 영향

평균 실행 시간 (1TB 데이터):

TPC-H:

- Q1: 1 초
- Q5: 5 초
- Q17: 30 초
- 평균: 10 초

TPC-DS:

- Q11: 15 초 (자체 조인)
- Q33: 45 초 (복잡한 마켓 바스켓)
- Q48: 90 초 (3 중 조인 + 서브쿼리)
- 평균: 40 초

→ 4 배 더 복잡한 워크로드

4.2 최적화 기법의 차이

TPC-H:

- 인덱싱으로 큰 성능 향상
- 조인 순서 최적화 중요
- 기존 DBMS 최적화로 충분

TPC-DS:

- 인덱싱 효과 제한적 (많은 조인으로 인해)
- 쿼리 재작성(rewrite) 필요
- 통계 기반 카디널리티 추정의 한계 드러남

66.5 벡터 검색 통합의 의미

5.1 TPC-DS 에 벡터 검색 추가의 현실성

현재 TPC-DS:

- 수치, 날짜, 범주형 데이터만
- 텍스트 검색은 정확히 매칭

현실의 데이터 웨어하우스:

- 고객 리뷰 (자유 텍스트)
- 상품 설명 (텍스트)
- 피드백 코멘트 (텍스트)
- 이미지 (비정형 데이터)

벡터 검색 추가의 필요성:

- "긍정적인 피드백을 남긴 고객"
- "특정 스타일의 옷을 찾는 고객"
- "유사한 리뷰를 남긴 다른 고객"

5.2 하이브리드 쿼리의 예

전통적 TPC-DS Q33 (마켓 바스켓):

```
SELECT product1, product2, COUNT(*)
FROM store_sales s1
JOIN store_sales s2
  ON s1.customer_id = s2.customer_id
WHERE s1.item_id < s2.item_id
GROUP BY product1, product2
```

확장된 버전 (벡터 검색 추가):

```
SELECT product1, product2, COUNT(*)
FROM store_sales s1
JOIN store_sales s2
  ON s1.customer_id = s2.customer_id
WHERE s1.item_id < s2.item_id
  AND item_description_embedding SIMILAR_TO
  (SELECT description_embedding FROM item WHERE id = ?)
GROUP BY product1, product2
```

의미: 특정 스타일의 상품을 자주 함께 구매하는 고객 찾기

66.6 현대적 확장 가능성

6.1 TPC-DS 의 한계

정적 데이터:

- 한 번 생성 후 수정 없음
- 실제 데이터는 지속적 변화

고정 쿼리:

- 99 개 쿼리는 모든 분석 패턴을 다 커버하지 못함
- 특히 텍스트, 이미지 검색은 완전히 빠짐

단일 도메인:

- 소매점만 다룸
- 금융, 의료, 전자상거래 등의 특수성 없음

6.2 벡터 검색 통합의 효과

다양한 데이터 타입 통합:

- 구조화(수치) + 반구조화(텍스트) + 비정형(이미지)

현실적 분석 쿼리:

- 단순 WHERE 필터 뿐 아니라
- 의미론적 유사도 기반 필터링

성능 이슈의 다양성:

- 전통적 쿼리 최적화 이슈
- 벡터 인덱싱의 효율성
- 혼합 조인 순서 최적화

66.7 본 논문과의 관계

Exqutor 의 TPC-DS 확장 고려 시:

- **현실성 증대:** TPC-H 보다 복잡한 TPC-DS 에 벡터 검색 추가로 더욱 현실적
- **다양한 쿼리 패턴:** 99 개 쿼리 중 어떤 것에 벡터 부분을 추가할 것인가
- **워크로드의 특성:** 마켓 바스켓, 시계열, 추천 등 다양한 비즈니스 분석
- **최적화 복잡도:** TPC-H 보다 높은 복잡도로 인한 새로운 최적화 도전

구체적으로:

- TPC-DS 의 자체 조인 쿼리에 텍스트 유사도 필터 추가
 - 고객 행동 분석 쿼리에 임베딩 기반 세분화 추가
 - 성능 모니터링이 더욱 복잡해지는 양상 분석
-

추가 제기 문제

1. **스케일별 특성 변화:** TPC-DS 를 1TB vs 10TB 로 실행할 때 벡터 검색의 성능 특성이 어떻게 달라지는가? 특히 메모리 부족 시나리오에서?
2. **다중 테이블 텍스트 필드:** TPC-DS 에는 여러 테이블에 텍스트 필드(customer_comments, item_description 등)가 있을 때, 어떤 필드를 임베딩할 것인가? 조인 순서에 미치는 영향은?
3. **계절성 데이터와 벡터 검색:** 계절성이 있는 데이터에서 일정 시간 전의 데이터로 학습한 임베딩 모델의 유효성은? 시간에 따른 드리프트(drift) 처리는?
4. **자체 조인과 벡터 필터:** TPC-DS 의 많은 쿼리가 자체 조인을 사용하는데, 유사 고객 또는 유사 상품을 찾는 벡터 필터를 추가할 때 조인 복잡도는 어떻게 변할 것인가?
5. **99 개 쿼리의 균등성:** TPC-DS 의 99 개 쿼리 중 벡터 검색이 의미 있는 이득을 줄 수 있는 쿼리는 몇 개인가? 특정 유형의 쿼리만 혜택을 받을 것인가?
6. **실시간 분석의 추가:** TPC-DS 는 배치 쿼리만 다루는데, 스트리밍 데이터와 벡터 검색의 결합은? 임베딩 모델 업데이트 빈도는?
7. **비용 모델의 재정의:** 벡터 데이터 추가로 인한 비용 모델(CPU, I/O, 메모리) 재정의는? 기존의 통계 기반 비용 모델이 벡터 부분도 적절히 예측할 수 있는가?